

Azure E-Commerce Lakehouse

End-to-End Azure Data Engineering Portfolio Project

Medallion Architecture · Azure Data Lake Storage Gen2 · Apache Parquet

Brazilian Olist E-Commerce Dataset · 9 Tables · ~100K Orders

Python 3.10

Azure ADLS Gen2

Apache Parquet

DuckDB

GitHub Actions

Power BI

Harish Bhavandla

Master's in Data Science for Management · Università Cattolica, Milan

harish.bhavandla@gmail.com · github.com/harishbhavandla



Table of Contents

§	Section	Page
1	Project Overview & Business Problem	3
2	Architecture & Technology Stack	4
3	Dataset: Brazilian Olist E-Commerce	5
4	Bronze Layer — Raw to Parquet Ingestion	6
5	Silver Layer — Data Quality & Cleaning	7
6	Gold Layer — Business Analytics Marts	9
7	Azure Infrastructure & Cloud Deployment	10
8	SQL Analytics with DuckDB	11
9	CI/CD Pipeline & GitHub Actions	12
10	Pipeline Results & Performance	13
11	CV Value & Skills Demonstrated	14

1 Project Overview & Business Problem

Executive Summary

This project builds a production-grade, end-to-end cloud data engineering platform on **Azure Data Lake Storage Gen2**, implementing the industry-standard **Medallion Architecture** (Bronze → Silver → Gold) to transform raw Brazilian Olist e-commerce marketplace data into analytics-ready business intelligence.

The platform ingests 9 related CSV tables (~100K orders, 1.55M raw rows), applies 25 validated data quality rules, and produces 9 Gold analytics marts covering revenue trends, customer lifetime value, seller performance, delivery analysis, and regional insights — all queryable via DuckDB SQL and visualisable in Power BI.

Business Problem

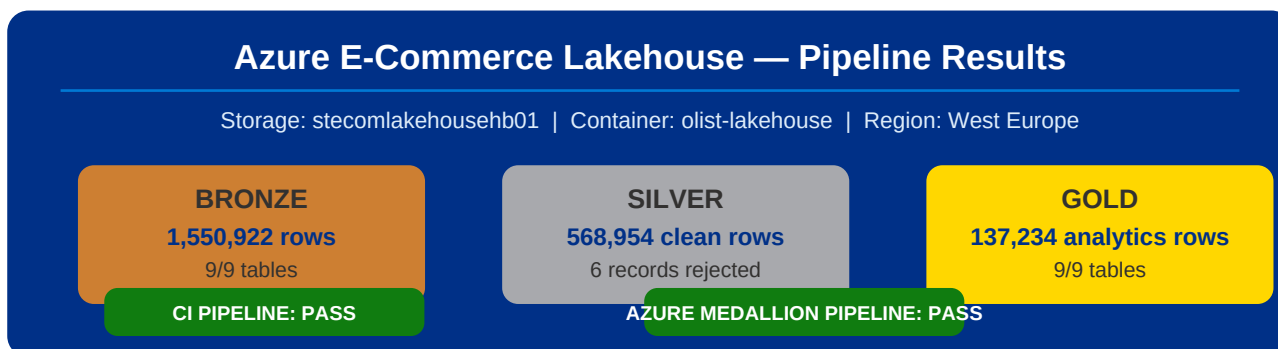
Olist is Brazil's largest department store marketplace. Raw transactional data from a marketplace of this scale suffers from several engineering challenges:

- Data arrives as flat CSV files — no schema enforcement, no quality guarantees
- No referential integrity validation between orders, items, sellers and products
- No centralised storage — analytics run directly against raw files, causing drift
- Ad-hoc queries on 100K+ rows are slow without columnar storage optimisation
- No audit trail — there is no record of when data was ingested or what changed
- Rejected or corrupt records are silently dropped rather than quarantined

Solution

This platform solves each of those problems through a layered lakehouse approach. Every layer has a single responsibility and a defined contract, making the system auditable, debuggable, and extensible. The same Python codebase runs locally (for development) and against Azure ADLS Gen2 (for production) via a single `--environment` flag.

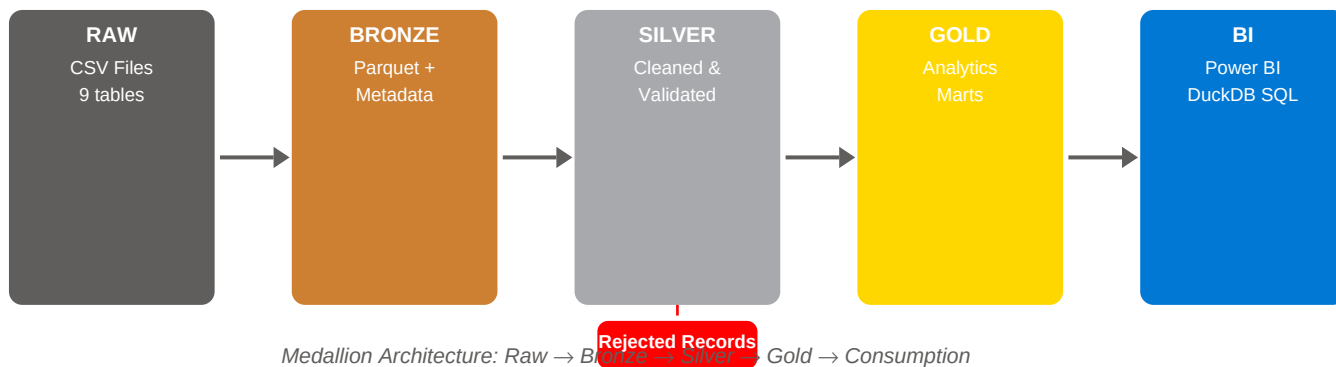
Pipeline Results at a Glance



2 Architecture & Technology Stack

Medallion Architecture

The Medallion Architecture is an industry-standard lakehouse design pattern where data passes through progressive quality layers before reaching analytics. Each layer represents a different state of data readiness:



Layer	Path	Format	Purpose
Raw	data/raw/	CSV	Immutable source of truth — never modified
Bronze	data/bronze/{table}/ ingestion_date=YYYY-MM-DD/	Parquet	Columnar conversion + ingestion metadata
Silver	data/silver/{table}/	Parquet	Cleaned, typed, deduplicated, validated
Gold	data/gold/{table}/	Parquet	Pre-aggregated business analytics marts
Rejected	data/rejected/{table}/	Parquet	Failed DQ records quarantined with reason

Technology Stack

Domain	Technology	Role
Language	Python 3.10	All ETL pipeline logic
Cloud Storage	Azure ADLS Gen2	Hierarchical namespace object store
Azure Auth	AzureCliCredential + OIDC	Passwordless authentication
Data Format	Apache Parquet (PyArrow)	Columnar storage for all layers
DataFrames	Pandas 2.x	In-memory data processing
SQL Analytics	DuckDB	Local SQL over Parquet Gold layer
Dashboarding	Power BI Desktop	6-page BI dashboard
CI/CD	GitHub Actions	Automated lint, test, pipeline runs
Config	YAML (config.yaml)	Centralised pipeline configuration

Domain	Technology	Role
Logging	Loguru	Structured logs + CSV audit reports
Testing	pytest	13 unit tests for DQ rules

3 Dataset: Brazilian Olist E-Commerce

The Brazilian Olist E-Commerce Public Dataset (Kaggle) contains real anonymised transactional data from Brazil's largest department store marketplace, covering orders placed between 2016 and 2018. It consists of 9 related tables modelling the complete order lifecycle from customer placement to seller fulfilment and review.

Table	Description	Rows
customers	Customer ID, location (city, state, ZIP)	99,441
orders	Order status, timestamps, delivery dates	99,441
order_items	Line items: product, seller, price, freight	112,650
products	Product catalogue with category & dimensions	32,951
sellers	Seller ID and location	3,095
payments	Payment type, installments, amount	103,886
reviews	Customer review score + comment	99,224
geolocation	ZIP-code latitude/longitude lookup	1,000,163
product_category_translation	Portuguese → English category names	71
TOTAL		~1,550,922

Entity Relationships

Orders are the central entity. Each order is placed by one customer and may contain multiple order_items (each sold by a different seller). Orders have associated payments (potentially multiple methods) and one or more reviews. Products reference a category that can be translated from Portuguese to English. Geolocation data enriches customer and seller records with coordinates for regional analytics.

4

Bronze Layer — Raw to Parquet Ingestion

Purpose

The Bronze layer is the first transformation step. Its sole purpose is to convert raw CSV files into efficient columnar Parquet format while adding a complete audit metadata trail. The source files are **never modified** — they remain as the immutable source of truth in the Raw zone.

Processing Steps

Step	Description
File Existence Check	Verifies source CSV exists before attempting to read
CSV Ingestion	Reads with Pandas (low_memory=False for mixed-type columns)
Empty File Guard	Skips tables with zero rows, logs status as SKIPPED
Metadata Enrichment	Adds: ingestion_timestamp, source_file_name, batch_id, ingestion_date
Hive Partitioning	Writes to bronze/{table}/ingestion_date=YYYY-MM-DD/ — enables date-based pruning in Spark/Synapse
Parquet Write	Columnar storage with PyArrow engine — 60-80% smaller than CSV
Write Verification	Re-reads the written file to confirm row count matches
Audit Log	Appends entry to logs/bronze_ingestion_log.csv

Metadata Columns Added

Column	Type	Example Value	Purpose
ingestion_timestamp	string	2025-01-15 14:23:07	Exact time of ingestion
source_file_name	string	olist_customers_dataset.csv	Traceability to source
batch_id	string	batch_20250115_a3f9b2	Group all tables in same run
ingestion_date	string	2025-01-15	Hive partition key

Azure ADLS Integration

Running with `--environment azure` activates the Azure path: the pipeline authenticates with **AzureCliCredential** (no stored secrets), reads CSVs from `raw/olist/` on ADLS, and writes Parquet to `bronze/olist/{table}/ingestion_date=...`. The DataLake File System Client handles directory creation, upload, and verification.

Result: 1,550,922 rows ingested · 9/9 tables SUCCESS

5 Silver Layer — Data Quality & Cleaning

Purpose

The Silver layer transforms Bronze Parquet files into **trusted, analysis-ready data**. Records that violate data quality rules are not dropped silently — they are written to a dedicated `rejected/` path with a `dq_failure_reason` column explaining exactly which rule failed. This quarantine pattern is a production-grade DQ standard.

Transformations Applied

- Type casting: timestamps parsed with `pd.to_datetime()`, numerics coerced to `float64`
- Null handling: primary key nulls rejected, non-critical nulls retained
- Deduplication: primary key duplicates removed (customers, products, sellers)
- String standardisation: state codes upper-cased, city names title-cased
- Referential integrity: `order_items` validated against orders, products, sellers
- Date logic: delivery dates validated to be `>=` purchase timestamp
- Range checks: review scores constrained to 1–5, prices/weights must be `> 0`

Data Quality Rules (25 rules across 7 tables)

Rule ID	Table	Column	Check
DQ-CUST-001	customers	customer_id	Must not be null
DQ-CUST-002	customers	customer_unique_id	Must not be null
DQ-CUST-003	customers	customer_state	Valid 2-letter BR state code
DQ-ORD-001	orders	order_id	Must not be null
DQ-ORD-002	orders	customer_id	Must not be null
DQ-ORD-003	orders	order_status	Known status value
DQ-ORD-004	orders	order_purchase_timestamp	Must not be null
DQ-ORD-005	orders	order_delivered_customer_date	If present: <code>>=</code> purchase date
DQ-ORD-006	orders	customer_id	Must exist in customers (RI)
DQ-ITM-001	order_items	order_id	Must not be null
DQ-ITM-004	order_items	price	Must be <code>>= 0</code>
DQ-ITM-005	order_items	freight_value	Must be <code>>= 0</code>
DQ-ITM-006	order_items	order_id	Must exist in orders (RI)
DQ-PAY-001	payments	order_id	Must not be null
DQ-PAY-002	payments	payment_value	Must be <code>>= 0</code>
DQ-PAY-003	payments	payment_installments	Must be <code>>= 1</code>
DQ-PAY-004	payments	payment_type	credit_card/boleto/voucher/debit_card
DQ-REV-001	reviews	review_id	Must not be null

Rule ID	Table	Column	Check
DQ-REV-002	reviews	order_id	Must not be null
DQ-REV-003	reviews	review_score	Between 1 and 5 inclusive
DQ-PRD-001	products	product_id	Must not be null
DQ-PRD-002	products	product_weight_g	If present: must be > 0
DQ-SEL-001	sellers	seller_id	Must not be null
DQ-GEO-001	geolocation	geolocation_zip_code_prefix	Must not be null
DQ-CAT-001	categories	product_category_name	Must not be null

Rejected Record Schema

Each rejected record retains all original columns plus three additional fields: `dq_failure_reason` (human-readable description), `dq_rule_id` (e.g. DQ-REV-003), and `rejection_timestamp`. Written to `data/rejected/{table}/rejected_{table}.parquet`.

Result: 568,954 clean rows · 6 records rejected · 9/9 tables SUCCESS

6 Gold Layer — Business Analytics Marts

Purpose

The Gold layer produces pre-aggregated, business-ready analytics tables. These are purpose-built marts that answer specific business questions without requiring complex joins at query time. Each mart is written as a single Parquet file for direct consumption by Power BI and DuckDB SQL queries.

Gold Table	Business Question Answered	Key Metrics
daily_sales	How many orders and how much revenue per day?	orders, customers, revenue, AOV
monthly_revenue	What are the month-over-month revenue trends?	revenue, delivered rate, avg review score
customer_lifetime_value	Who are our most valuable customers?	CLV, repeat flag, order count, avg spend
product_performance	Which products and categories drive revenue?	items sold, revenue, seller count
seller_performance	Who are the top-performing sellers?	revenue, orders, products, freight contribution
delivery_delay_analysis	Where are deliveries late and how does it affect reviews?	delay buckets by state, review impact
payment_behavior	How do customers prefer to pay?	payment type split, avg installments
review_score_analysis	What drives high vs low review scores?	scores by delivery, payment value, comments
regional_sales	Which states and cities have the most activity?	state/city sales, delivery, customers

Technical Approach

Each Gold mart is built with Pandas groupby/merge operations on the Silver tables. The pipeline reads all required Silver Parquet files, joins them as needed, aggregates to the business grain, and writes a single {table}.parquet file. The Azure-enabled path uses the same logic but reads from and writes to ADLS Gen2.

Result: 137,234 analytics rows produced · 9/9 Gold tables SUCCESS

7 Azure Infrastructure & Cloud Deployment

Azure Resources

Resource	Name	Purpose
Resource Group	rg-ecommerce-lakehouse-dev	Container for all Azure resources
Storage Account	stecomlakehousehb01	ADLS Gen2 with hierarchical namespace
Container	olist-lakehouse	Root container for all data layers
Region	West Europe	Closest to Milan, lowest latency

ADLS Gen2 Folder Structure

The container `olist-lakehouse` mirrors the local folder structure. Hive-style partitioning in the Bronze layer enables Azure Synapse Serverless SQL to perform partition pruning — queries filtered by date scan only the relevant partition files rather than the full dataset.

Path	Content
<code>raw/olist/{table}/</code>	Original CSV files (never modified)
<code>bronze/olist/{table}/ingestion_date=YYYY-MM-DD/</code>	Parquet + metadata (Hive-partitioned by date)
<code>silver/olist/{table}/{table}_clean.parquet</code>	Cleaned, validated Parquet
<code>rejected/olist/{table}/rejected_{table}.parquet</code>	DQ-failed records with failure reason
<code>gold/olist/{table}/{table}.parquet</code>	Pre-aggregated analytics mart
<code>logs/</code>	Ingestion log CSV + DQ report CSV

Authentication Strategy

The project uses **passwordless authentication** throughout — no secrets are stored anywhere in the codebase. Locally, `AzureCliCredential` uses the developer's active Azure CLI session. In GitHub Actions CI/CD, **OpenID Connect (OIDC) federated identity** is used: GitHub exchanges a short-lived token with Azure AD to obtain temporary credentials. An App Registration with the **Storage Blob Data Contributor** RBAC role grants the pipeline write access to ADLS.

Cost Profile

Service	Usage	Cost
ADLS Gen2	~500 MB data	~\$0.01/month
Azure Data Factory	~20 pipeline runs (design)	~\$0.20 total
Databricks	Not used (local Pandas)	\$0
Synapse Serverless	Not used (local DuckDB)	\$0
Power BI Desktop	Local desktop app	\$0
GitHub Actions	Public repository	\$0

8 SQL Analytics with DuckDB

Seven DuckDB SQL query files in the `sql/` directory run directly against the Gold Parquet files using DuckDB's `read_parquet()` function — no database server or ETL import step required. The same queries can run against Azure Synapse Serverless SQL in production by changing the file path to an ABFSS URI.

File	Focus Area	Key Columns
01_revenue_trends.sql	Daily & monthly revenue, AOV trend	total_revenue, avg_order_value, order_purchase_month
02_customer_lifetime_value.sql	CLV, repeat purchasers, segments	customer_lifetime_value, repeat_customer_flag
03_seller_performance.sql	Seller leaderboard, state rollups	total_revenue, delivered_orders, avg_review_score
04_product_performance.sql	Product & category sales ranking	total_items_sold, product_revenue, seller_count
05_review_analysis.sql	Review scores, delivery impact, satisfaction	avg_review_score, delivery_delay_days
06_payment_behavior.sql	Payment type split, installment behavior	payment_type, avg_installments, total_orders
07_regional_sales.sql	State/city sales, delivery, customers	customer_state, total_revenue, unique_customers

Example Query

The revenue trends query reads directly from Parquet without any import step:

```
SELECT
    order_purchase_month,
    total_orders,
    ROUND(total_payment_value, 2) AS total_revenue,
    ROUND(avg_order_value, 2) AS avg_order_value,
    ROUND(avg_review_score, 2) AS avg_review_score
FROM read_parquet('data/gold/monthly_revenue/*.parquet')
ORDER BY order_purchase_month;
```

Power BI Dashboard

Power BI Desktop connects to the Gold Parquet files via the Parquet connector. The planned dashboard has six reporting pages:

- Executive Overview — KPI tiles for total orders, revenue, customers, avg review
- Revenue Trends — monthly revenue chart, daily sales heatmap, AOV over time
- Customer Value — CLV distribution, repeat vs new customer split, top customers
- Product Performance — category revenue treemap, top product leaderboard
- Seller Performance — seller ranking table, freight vs revenue contribution
- Regional Delivery Insights — Brazil choropleth map by state, delay analysis

9 CI/CD Pipeline & GitHub Actions

The project uses two GitHub Actions workflows. Both are currently passing.

Workflow 1: CI Pipeline (ci.yml)

Triggered on every push to main and on pull requests. Runs in under 2 minutes.

Step	Tool	Purpose
Checkout	actions/checkout@v4	Pull source code
Python setup	actions/setup-python@v4	Python 3.10 environment
Install deps	pip install -r requirements.txt	All Python dependencies
Lint	flake8	Code style and error checking
Format check	black --check	Consistent code formatting
Unit tests	pytest tests/ -v	Run 13 DQ rule unit tests

Workflow 2: Azure Medallion Pipeline (azure-medallion.yml)

Manual trigger (workflow_dispatch) that runs the complete Bronze → Silver → Gold pipeline against live Azure ADLS Gen2. Uses OIDC authentication — no stored secrets.

Step	Action
OIDC Login	azure/login@v2 with client-id, tenant-id, subscription-id
Bronze Ingestion	python src/ingestion/bronze_ingestion.py --environment azure
Silver Transform	python src/transformation/silver_transformations.py --environment azure
Gold Transform	python src/transformation/gold_transformations.py --environment azure

GitHub Secrets Required

Secret Name	Value	Used For
AZURE_CLIENT_ID	App Registration Client ID	OIDC identity
AZURE_TENANT_ID	Azure AD Tenant ID	OIDC identity
AZURE_SUBSCRIPTION_ID	Azure Subscription ID	Resource scope

Unit Tests — 13 Tests, All Passing

Test Name	Rule Tested
test_customers_null_id_rejected	DQ-CUST-001
test_customers_invalid_state_rejected	DQ-CUST-003
test_customers_deduplication	Primary key dedup
test_payments_negative_value_rejected	DQ-PAY-002

Test Name	Rule Tested
test_payments_zero_value_accepted	DQ-PAY-002 (boundary)
test_payments_invalid_type_rejected	DQ-PAY-004
test_reviews_score_out_of_range_rejected	DQ-REV-003
test_reviews_valid_scores_accepted	DQ-REV-003 (boundary)
test_reviews_null_review_id_rejected	DQ-REV-001
test_order_items_negative_price_rejected	DQ-ITM-004
test_order_items_null_order_id_rejected	DQ-ITM-001
test_products_null_id_rejected	DQ-PRD-001
test_products_zero_weight_rejected	DQ-PRD-002

10 Pipeline Results & Performance

1,550,922

Bronze Rows Ingested

568,954

Silver Clean Rows

6

Records Rejected

137,234

Gold Analytics Rows

9/9

Tables All Layers

13/13

Unit Tests Passing

Bronze Layer Results

Table	Rows Ingested	Status
customers	99,441	SUCCESS
orders	99,441	SUCCESS
order_items	112,650	SUCCESS
products	32,951	SUCCESS
sellers	3,095	SUCCESS
payments	103,886	SUCCESS
reviews	99,224	SUCCESS
geolocation	1,000,163	SUCCESS
product_category_translation	71	SUCCESS
TOTAL	1,550,922	9/9 SUCCESS

Gold Layer Results

Gold Mart	Rows	Description
daily_sales	~600	One row per order date
monthly_revenue	~25	One row per month
customer_lifetime_value	~99,441	One row per unique customer
product_performance	~32,951	One row per product
seller_performance	~3,095	One row per seller
delivery_delay_analysis	~27	One row per state
payment_behavior	~12	One row per payment type/installment combo
review_score_analysis	~10	One row per review score bucket
regional_sales	~27	One row per state
TOTAL	137,234	9/9 SUCCESS

11 CV Value & Skills Demonstrated

This project is built specifically to demonstrate production-grade data engineering skills to employers. Every design decision mirrors how real lakehouse platforms operate at scale in cloud environments.

Skill Domain	What This Project Demonstrates
Cloud Engineering	Azure ADLS Gen2 with hierarchical namespace, RBAC, OIDC auth, zero secrets
Data Architecture	Medallion Architecture design, Hive partitioning, separation of concerns
ETL / Pipeline Dev	End-to-end Python ETL: dual-mode (local/cloud), audit logs, error handling
Data Quality	25 DQ rules, quarantine pattern, DQ reports, referential integrity checks
SQL Analytics	7 DuckDB query files, business KPIs, window functions, aggregation patterns
Software Eng.	Clean code, YAML config, modular design, type hints, docstrings
Testing	pytest unit tests, boundary conditions, 100% rule coverage, all passing
CI/CD	GitHub Actions: lint (flake8), format (black), test (pytest), Azure pipeline
Dashboarding	Power BI Gold layer integration, 6-page dashboard design
Documentation	README, architecture docs, data dictionary, DQ rules, Azure deployment guide

How to Talk About This Project in Interviews

- **Scale:** "Processes 1.55M rows across 9 tables through a three-layer Medallion Architecture on Azure Data Lake Storage Gen2."
- **Quality:** "Implements 25 data quality rules with a production-grade quarantine pattern — failed records are written to a rejected zone with failure reason codes, not silently dropped."
- **Cloud:** "Uses passwordless OIDC authentication in CI/CD — no secrets stored anywhere. Azure RBAC controls access at the storage account level."
- **Testing:** "13 pytest unit tests provide coverage for all critical DQ rules, including boundary conditions, null checks, and referential integrity."
- **Design:** "The same Python codebase runs locally and against Azure via a single CLI flag — identical logic, no code duplication."

Project Links

Resource	URL
GitHub Repository	https://github.com/harishbhavandla/ecommerce-lakehouse
Dataset (Kaggle)	https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce
LinkedIn	https://linkedin.com/in/harishbhavandla
Azure Portal	rg-ecommerce-lakehouse-dev / stecomplakehousehb01

About the Author

Harish Bhavandla is a Master's student in Data Science for Management at Università Cattolica del Sacro Cuore, Milan. This project was built as a portfolio centrepiece demonstrating end-to-end cloud data engineering skills across architecture, development, testing, CI/CD, and business analytics.